Stolpe,
Titel Kurz

Der Lange Titel der Arbeit die zu Schreiben Ist Ein Untertitel ist Optional

The Long Title of the Work to Be Done This is Optionally Optional

by

Daniel Stolpe

A thesis submitted to the
Hasso Plattner Institute
at the University of Potsdam, Germany
in partial fulfillment of the requirements for the degree of

Master of Science in IT-Systems Engineering

Supervisors

Prof. Dr. Robert Hirschfeld
Dr. John Doe

Software Architecture Group
Hasso Plattner Institute
University of Potsdam, Germany

September 18, 2018

Abstract

The english abstract.

Zusammenfassung

Die Zusammenfassung auf deutsch.

Acknowledgments

I owe everything to my cat.

Contents

1. Introduction **1**
1.1. Challanges of developing your own language . . . 1
1.2. Language implementations - performance vs. tooling support . . . 1
1.3. Structur of this Thesis . . . 1

2. Context **3**
2.1. Language Server Protocol . . . 3
2.2. Graal and Truffle . . . 3

3. Solution **5**
3.1. How to get the relevant information out of Graal/Truffle . . . 5
3.1.1. Parsing code . . . 5
3.1.2. Mapping editor cursor position to Truffle source-section and node . . . 5
3.1.3. Instrumentation . . . 5
3.1.4. Challanges due to Truffles focus on performance . . . 5
3.2. Type harvesting evaluation strategies . . . 5
3.2.1. Literals . . . 5
3.2.2. Coverage data . . . 5
3.2.3. Run-to-section . . . 5
3.2.4. Globals . . . 5
3.3. LSP features overview table . . . 5

4. Implementation **7**
4.1. The lsp4j library . . . 8
4.2. Architecture . . . 8
4.2.1. Truffle-API + Graal-SDK . . . 8
4.2.2. Launcher, Instrument, Server . . . 8
4.2.3. Thread with entered Context . . . 8
4.3. What a TruffleLanguage needs to implement . . . 8
4.3.1. Expression- and Statement-Tag . . . 8
4.3.2. findSourceLocation() . . . 8
4.3.3. findTopScopes() . . . 8
4.3.4. findLocalScopes() . . . 8
4.3.5. findMetaObject() . . . 8
4.3.6. getCompletionTriggerCharacters() . . . 8
4.3.7. boxPrimitive() . . . 8
4.3.8. parseInline() . . . 8
4.3.9. GetSignature custom message . . . 8

4.4. LSP features implementation TextDocumentService 8
4.4.1. completion 8
4.4.2. hover 8
4.4.3. signatureHelp 8
4.4.4. definition 8
4.4.5. typeDefinition 8
4.4.6. implementation 8
4.4.7. references 8
4.4.8. documentHighlight 8
4.4.9. documentSymbol 8
4.4.10. codeAction 8
4.4.11. codeLense 8
4.4.12. formatting 8
4.4.13. rangeFormatting 8
4.4.14. onTypeFormatting 8
4.4.15. rename 8
4.4.16. didOpen 8
4.4.17. didChange 8
4.4.18. didClose 8
4.4.19. didSave 8
4.4.20. willSave 8
4.4.21. willSaveWaitUntil 8
4.4.22. documentLink 8
4.4.23. documentLinkResolve 8
4.4.24. documentColor 8
4.4.25. colorPresentation 8
4.5. LSP features implementation WorkspaceService 8
4.5.1. executeCommand 8
4.5.2. symbol 8
4.5.3. didChangeConfiguration 8
4.5.4. didChangeWatchedFiles 8
4.5.5. didChangeWorkspaceFolders 8

5. Evaluation **9**
5.1. Between IDE and Language Workbench 9
5.2. Completion response time 9
5.3. Limitations 9
5.3.1. VSCode: Need to define supported languages in extension . . 9
5.3.2. VSCode: Syntax highlighting defined at client-side 9

6. Related Work **11**
6.1. Graal 11
6.2. Squimera 11
6.3. Language Workbenches 11

7. Conclusion **13**

A. First Unimportant stuff. **17**

List of Figures

List of Tables

List of Listings

List of Abbreviations

API application programming interface

1. Introduction

1.1. Challanges of developing your own language

For a simple example, consider if we want to rename the properties on the target class in my simple example. With a first class modern IDE, automatic refactorings for renaming are habitual. But such a rename won't propagate into the DSL. There is what I'll call a symbolic barrier between the world of C# and the file mapping DSL. We can translate our mapping into C#, but the barrier limits our ability to manipulate the overall program. This lack of integration hits us in lots of ways with tooling. Firstly - how do we edit our DSL? A text editor will do the job - but modern IDEs increasingly make text editors look primitive. I should get a pop-up list and completion on the field names, red squiggles if the character ranges overlap. But to do this I need an editor that understands the semantics of my DSL. Maybe I can live without a semantic editor. But then think about debugging. My debugger can step into the C# translations, but can't get into the true source itself. What I really would like is a full-blown IDE for my DSL. In the days of text editors and simple debuggers this wasn't a big issue - but we now live in a post-IntelliJ world.

1.2. Language implementations - performance vs. tooling support

1.3. Structur of this Thesis

2. Context

2.1. Language Server Protocol

2.2. Graal and Truffle

3. Solution

3.1. How to get the relevant information out of Graal/Truffle

3.1.1. Parsing code

3.1.2. Mapping editor cursor position to Truffle source-section and node

3.1.3. Instrumentation

3.1.4. Challanges due to Truffles focus on performance

AST nodes lack relevant information, e.g. for refactoring. The language parsers are designed as fail-on-first-error. Only first syntax error is reported.

3.2. Type harvesting evaluation strategies

3.2.1. Literals

3.2.2. Coverage data

3.2.3. Run-to-section

3.2.4. Globals

3.3. LSP features overview table

4. Implementation

4.1. The lsp4j library

4.2. Architecture

4.2.1. Truffle-API + Graal-SDK

4.2.2. Launcher, Instrument, Server

4.2.3. Thread with entered Context

4.3. What a TruffleLanguage needs to implement

4.3.1. Expression- and Statement-Tag

4.3.2. findSourceLocation()

4.3.3. findTopScopes()

4.3.4. findLocalScopes()

4.3.5. findMetaObject()

4.3.6. getCompletionTriggerCharacters()

4.3.7. boxPrimitive()

4.3.8. parseInline()

4.3.9. GetSignature custom message

4.4. LSP features implementation TextDocumentService

4.4.1. completion

4.4.2. hover

4.4.3. signatureHelp

4.4.4. definition

4.4.5. typeDefinition

4.4.6. implementation

4.4.7. references

4.4.8. documentHighlight

4.4.9. documentSymbol

5. Evaluation

5.1. Between IDE and Language Workbench

5.2. Completion response time

5.3. Limitations

5.3.1. VSCode: Need to define supported languages in extension

5.3.2. VSCode: Syntax highlighting defined at client-side

6. Related Work

6.1. Graal

[4]

6.2. Squimera

[3] [2]

6.3. Language Workbenches

[1]

7. Conclusion

Bibliography

[1] Sebastian Erdweg, Tijs Van Der Storm, Markus Völter, Meinte Boersma, Remi Bosman, William R Cook, Albert Gerritsen, Angelo Hulshout, Steven Kelly, Alex Loh, et al. "The state of the art in language workbenches". In: *International Conference on Software Language Engineering*. Springer. 2013, pp. 197–217.

[2] Fabio Niephaus, Tim Felgentreff, Tobias Pape, and Robert Hirschfeld. "Squeak Makes a Good Python Debugger: Bringing Other Programming Languages Into Smalltalk's Tools". In: *Companion to the first International Conference on the Art, Science and Engineering of Programming*. ACM. 2017, p. 27.

[3] Fabio Niephaus, Tim Felgentreff, Tobias Pape, Robert Hirschfeld, and Marcel Taeumel. "Live Multi-language Development and Runtime Environments". In: *arXiv preprint arXiv:1803.10200* (2018).

[4] Chris Seaton, Michael L Van De Vanter, and Michael Haupt. "Debugging at full speed". In: *Proceedings of the Workshop on Dynamic Languages and Applications*. ACM. 2014, pp. 1–13.

Appendix A.

First Unimportant stuff.

Hello, here is some text without a meaning. This text should show what a printed text will look like at this place. If you read this text, you will get no information. Really? Is there no information? Is there a difference between this text and some nonsense like “Huardest gefburn”? Kjift – not at all! A blind text like this gives you information about the selected font, how the letters are written and an impression of the look. This text should contain all letters of the alphabet and it should be written in of the original language. There is no need for special content, but the length of words should match the language.

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbständig verfasst sowie keine anderen Quellen und Hilfsmittel als die angegebenen benutzt habe.

Berlin, den 18. September 2018

Daniel Stolpe

Todo list